ESCUELA POLITÉCNICA NACIONAL
ESCUELA DE FORMACIÓN DE TECNÓLOGOS (ESFOT)
Tecnología Superior en Desarrollo de Software

PERIODO ACADÉMICO: 2026-B
ASIGNATURA: Aplicaciones Móviles
PONDERACIÓN: 4.0 / 20 puntos

INFORME DE CORRECCIÓN Y MEJORA: EVALUACIÓN TEÓRICA

Revisión Técnica Exhaustiva, Análisis Crítico y Fundamentación de Respuestas

Estudiante: Jimy Lucero	**Docente:** Ing. Vanessa Guevara, MsC.	**Fecha de Corrección:** 4 de noviembre de 2026
Repositorio GitHub: JimSul16/IniMoviles2026B	**Rama de Corrección:** correccion-mejoras	**Estado:** Corregido al 100% (10/10)

Presentación del documento: El presente informe contiene la revisión detallada y corrección formal de cada una de las preguntas de la evaluación teórica del Primer Bimestre. Para cada ítem se expone: (1) el enunciado original con su puntaje, (2) la respuesta inicial emitida durante la evaluación, (3) el análisis crítico y diagnóstico de las imprecisiones o vacíos técnicos, y (4) la respuesta corregida con su respectiva fundamentación técnica basada en la documentación oficial de Ionic, Angular, Capacitor y especificaciones HTTP.

PREGUNTA 1

Puntaje: 1,00 pt

Enunciado: *¿Cuáles son los comandos o pasos requeridos para crear e inicializar un proyecto en Ionic utilizando la plantilla de pestañas (tabs) y componentes standalone con Angular?*

Respuesta emitida en la prueba:

«mkdir proyecto-medidores
cd proyecto-medidores
npm install
npx ionic start»

Diagnóstico del error y aspectos a mejorar:

La secuencia propuesta es incorrecta y redundante. Crear carpetas manuales con mkdir y cd es innecesario porque la CLI de Ionic genera automáticamente el directorio raíz. Ejecutar npm install en una carpeta vacía sin package.json genera un error de npm. Además, invocar npx ionic start sin argumentos inicia un asistente interactivo manual que no automatiza los parámetros solicitados.

Respuesta Corregida y Fundamentación Técnica:

El comando estándar y directo en una sola línea provisto por la CLI oficial de Ionic es:
ionic start proyecto-medidores tabs --type=angular --standalone
O utilizando el ejecutor npx sin instalación global previa:
npx @ionic/cli start proyecto-medidores tabs --type=angular --standalone

Desglose y justificación técnica de los parámetros:
• **proyecto-medidores:** Nombre del proyecto; la CLI crea automáticamente esta carpeta en el directorio actual y configura el package.json.
• **tabs:** Plantilla preconfigurada que estructura una interfaz móvil con barra de navegación inferior (ion-tabs, ion-tab-bar, ion-tab-button).
• **--type=angular:** Especifica a Angular como framework subyacente (frente a opciones como React o Vue).
• **--standalone:** Configura el proyecto con la arquitectura moderna de Angular basada en componentes, directivas y pipes independientes (*Standalone Components*), prescindiendo por completo de los módulos tradicionales NgModule.
La CLI se encarga automáticamente de descargar la plantilla, resolver el enrutamiento y ejecutar npm install sin intervención manual adicional.

PREGUNTA 2

Puntaje: 1,00 pt

Enunciado: *Explique qué es ngOnInit().*

Respuesta emitida en la prueba:

«Es un iniciador de la aplicacion»

Diagnóstico del error y aspectos a mejorar:

Respuesta excesivamente simplista e imprecisa. Confunde el inicio global de la aplicación (proceso de arranque o *bootstrapping* en main.ts) con un método específico del ciclo de vida que pertenece individualmente a cada componente de Angular.

Respuesta Corregida y Fundamentación Técnica:

ngOnInit() es un **gancho del ciclo de vida (*lifecycle hook*)** de Angular definido en la interfaz OnInit.

Momento de ejecución y propósito técnico:
1. Se ejecuta **una sola vez** inmediatamente después de que Angular ha instanciado el componente, ha resuelto la inyección de dependencias y ha inicializado todas las propiedades de entrada vinculadas (@Input() o la función input()).
2. **Diferencia con el constructor:** El constructor() pertenece a la clase de TypeScript y debe utilizarse únicamente para la inyección de dependencias liviana. No debe contener lógica de negocio ni peticiones HTTP. ngOnInit() es el lugar arquitectónicamente correcto para ejecutar inicializaciones complejas, suscribirse a Observables, configurar Signals o disparar peticiones HTTP para obtener datos iniciales de una API.
3. **Comportamiento en Ionic:** En aplicaciones Ionic, debido a que el enrutador puede mantener páginas cacheadas en el DOM al navegar, ngOnInit() solo se ejecuta la primera vez que la página se crea. Por ello, se complementa con los ganchos visuales propios de Ionic como ionViewWillEnter() (que se dispara cada vez que la pantalla vuelve a ser visible para el usuario) y ionViewWillLeave().

PREGUNTA 3

Puntaje: 1,00 pt

Enunciado: *¿Para qué sirve ion-infinite-scroll? Explique qué ocurre cuando se dispara ionInfinite y por qué debe finalizarse la operación de carga.*

Respuesta emitida en la prueba:

«Es para poder dentro de una aplicacion al hacer scroll como por ejemplo dentro de una aplicacion de viajes es hacer que al momento de bajar no sea nesesario cargar todo el catalogo y tenerlo ya listo»

Diagnóstico del error y aspectos a mejorar:

Describe la intuición del scroll infinito, pero omite los aspectos técnicos centrales de la pregunta: no explica qué evento emite Ionic en runtime, qué cambios de estado interno ocurren y por qué la invocación de complete() es mandatoria.

Respuesta Corregida y Fundamentación Técnica:

ion-infinite-scroll es un componente de Ionic diseñado para implementar **carga progresiva y paginación incremental bajo demanda** conforme el usuario se desplaza verticalmente hacia el final de la lista.

1. ¿Qué ocurre cuando se dispara ionInfinite?
Cuando el usuario se acerca a la parte inferior del contenedor (al alcanzar el umbral o threshold configurado, típicamente 15% o 100px), el componente emite el evento personalizado (ionInfinite). En ese momento, Ionic activa un indicador visual animado (spinner dentro de ion-infinite-scroll-content) y **bloquea la detección de nuevos eventos de scroll** para prevenir condiciones de carrera y peticiones HTTP duplicadas.

2. ¿Por qué es obligatorio finalizar la operación con complete()?
Una vez que la aplicación recibe la nueva página de datos desde la API y la concatena a la lista existente, es **estrictamente necesario invocar event.target.complete()** (preferiblemente en el bloque finally de la promesa o del Observable).
• **Si no se llama a complete():** El componente permanece atrapado indefinidamente en estado 'loading', el spinner nunca desaparece y el listener de scroll no se reactiva, imposibilitando cargar páginas futuras.
• **Fin real de catálogo:** Cuando la API ya no tiene más registros (ej. items.length < pageSize), se debe establecer [disabled]='true' para evitar peticiones vacías adicionales.

PREGUNTA 4

Puntaje: 1,00 pt

Enunciado: *Explique qué es Capacitor y cuál es su función en Ionic. Mencione dos plugins revisados en clase e indique para qué sirven.*

Respuesta emitida en la prueba:

«Ejemplos: capacitor/assets y capacitor/camera. Su funcion en Ionic es de proveer una herramienta mas para poder hacer codigo fuente en casos como de logica o de construccion de interfaces.»

Diagnóstico del error y aspectos a mejorar:

Grave error conceptual. Capacitor no provee herramientas de interfaz ni escribe código de lógica. Además, @capacitor/assets es una herramienta de soporte por consola para generar iconos y pantallas de carga (splash screen), no un plugin nativo de runtime revisado en clase para el funcionamiento de la app.

Respuesta Corregida y Fundamentación Técnica:

Capacitor es el **runtime multiplataforma nativo y puente de integración (*native bridge*)** oficial de código abierto creado por Ionic.

Función fundamental en Ionic:
1. **Empaquetamiento nativo:** Aloja la aplicación web (desarrollada con HTML, CSS, JavaScript/TypeScript y Angular) dentro de una vista web nativa optimizada (Android Chromium WebView en Android y WKWebView en iOS).
2. **Puente de comunicación (*Bridge*):** Proporciona una capa de abstracción estandarizada que comunica el código web con las APIs de bajo nivel y hardware del sistema operativo móvil a través de llamadas asíncronas seguras.

Dos plugins nativos oficiales revisados en clase:
• **@capacitor/camera:** Permite capturar fotografías mediante la cámara física del dispositivo o seleccionarlas de la galería de medios. Gestiona los permisos nativos en tiempo de ejecución (CAMERA, READ_MEDIA_IMAGES) y entrega la imagen en Base64, Uri o DataUrl.
• **@capacitor/geolocation:** Accede al hardware de geolocalización satelital (GPS) y redes del dispositivo móvil, permitiendo obtener las coordenadas geográficas precisas (latitud, longitud, altitud y precisión) para registrar la ubicación exacta de campo.
(Plugin complementario visto en clase: @capacitor/preferences para almacenamiento clave-valor persistente).

PREGUNTA 5

Puntaje: 1,00 pt

Enunciado: *Explique cómo se obtiene una fotografía mediante @capacitor/camera. ¿Qué es Base64?*

Respuesta emitida en la prueba:

«Base64 principalmente es para poder digamos cifrar informacion, cuando se toma una fotografia por medio de la aplicacion movil viene codificada en Base64 y con eso mismo se decodifica y se transforma a un formato .jpg»

Diagnóstico del error y aspectos a mejorar:

Error conceptual crítico: **Base64 NO es un algoritmo de cifrado ni de seguridad**; cualquier entidad puede decodificarlo sin claves. Además, no detalló el método de captura de la API de Capacitor.

Respuesta Corregida y Fundamentación Técnica:

1. Cómo se obtiene una fotografía con @capacitor/camera:
Se importa la clase estática Camera y las opciones tipadas desde @capacitor/camera. Se invoca el método asíncrono Camera.getPhoto() configurando la fuente y el tipo de resultado:

```
const foto = await Camera.getPhoto({
  quality: 85,
  allowEditing: false,
  resultType: CameraResultType.Base64, // o DataUrl
  source: CameraSource.Prompt // Pregunta si desea Cámara o Galería
});
const base64Data = foto.base64String;
```

El plugin se encarga automáticamente de solicitar los permisos al usuario y abrir la interfaz de cámara nativa.

2. ¿Qué es Base64 exactamente?
Base64 es un **esquema de codificación de binario a texto (RFC 4648)**, no un algoritmo de encriptación o cifrado.
• **Propósito:** Su función es representar datos binarios crudos (como imágenes JPEG/PNG, audios o archivos) mediante un conjunto de 64 caracteres ASCII seguros e imprimibles (A-Z, a-z, 0-9, +, / y = como relleno o *padding*).
• **Uso en aplicaciones móviles:** Permite incrustar y transportar imágenes directamente dentro de cargas útiles JSON de peticiones HTTP REST (ej. campo foto en MockAPI) o mediante Data-URIs (...) sin requerir almacenamiento multipart inmediato.
• **Sobrecarga:** Cada 3 bytes de datos binarios (24 bits) se representan con 4 caracteres de texto, generando un incremento predecible de aproximadamente el 33% en el tamaño de los datos.

PREGUNTA 6

Puntaje: 1,00 pt

Enunciado: *¿Para qué se utiliza @capacitor/preferences? Describa un caso en el que conviene guardar información localmente en vez de volver a solicitarla a una API.*

Respuesta emitida en la prueba:

«Caso: Conviene guardar datos localmente por ejemplo en una base de datos relacional o no relacional cuando no se puede tener esa APi siempre localmente puede ser por un tema de puertos o de incluirla en la construccion de la aplicacion movil.»

Diagnóstico del error y aspectos a mejorar:

Confunde el almacenamiento clave-valor ligero con bases de datos SQL o NoSQL. Menciona temas de puertos inexistentes en clientes móviles y no explica los métodos ni la naturaleza del plugin.

Respuesta Corregida y Fundamentación Técnica:

@capacitor/preferences es el plugin nativo de Ionic para almacenar **datos livianos de forma persistente en pares Clave-Valor (*key-value storage*)** en el propio dispositivo.

Funcionamiento técnico:
Utiliza los mecanismos nativos más rápidos y eficientes de cada plataforma: SharedPreferences en Android, UserDefaults en iOS y localStorage en la Web. Ofrece una API asíncrona no bloqueante: Preferences.set(), Preferences.get(), Preferences.remove() y Preferences.clear().

Casos concretos donde conviene guardar localmente:
1. **Tokens de sesión y autenticación (JWT):** Almacenar el token del inspector permite validar su identidad sin solicitar credenciales ni peticiones de login cada vez que la app se abre.
2. **Preferencias de usuario y configuración:** Guardar el tema seleccionado (modo oscuro/claro), idioma preferido o unidades métricas.
3. **Seguimiento personal de incidencias (Requisito R1):** Almacenar los IDs de incidencias marcadas para seguimiento prioritario en el dispositivo del inspector. Evita saturar la base de datos de la API municipal con selecciones que son exclusivas del trabajo de campo personal del inspector.
4. **Borradores y recuperación ante cierres inesperados (Requisito R8):** Si la batería se agota o la app se cierra en medio del llenado de una inspección, los datos se conservan en Preferences para permitir 'Continuar o Descartar' al reiniciar.

PREGUNTA 7

Puntaje: 1,00 pt

Enunciado: *Explique qué hacen ionic build, ionic cap add android, ionic cap sync android e ionic cap open android. ¿Con qué herramienta se genera posteriormente un APK instalable?*

Respuesta emitida en la prueba:

«Los comandos de 'ionic build,ionic cap add android' y demas son posteriores cuando ya se finalizo el proyecto y deseamos ver si corre en nuestro celular Android o en un emulador como por ejemplo 'Bluestacks' o en el mismo android studio, la herramienta se puede generar mediante Android Studio o mediante terminal instalando una herramienta llamada Gradle que viene junto a la instalacion de Java. Ejemplo cd Appmovil /gradle»

Diagnóstico del error y aspectos a mejorar:

Respuesta imprecisa y desorganizada. No diferencia las acciones individuales de cada uno de los 4 comandos. Afirma erróneamente que Gradle viene incluido dentro de Java y propone comandos ficticios para compilar.

Respuesta Corregida y Fundamentación Técnica:

Explicación técnica de cada comando del flujo Capacitor:

1. **ionic build (o ng build):** Compila y empaqueta el código fuente Angular/TypeScript, procesa hojas de estilo SCSS y optimiza los activos web de producción en el directorio de salida www/.
2. **ionic cap add android:** Crea e inicializa el proyecto nativo de Android en el subdirectorio /android. Genera la estructura completa con build.gradle, AndroidManifest.xml y la actividad principal MainActivity.java. Se ejecuta únicamente una vez.
3. **ionic cap sync android:** Ejecuta en una sola orden la sincronización completa del proyecto nativo:
   - cap copy: Copia los activos web recién compilados desde www/ hacia android/app/src/main/assets/public/.
   - cap update: Lee las dependencias y registra los plugins nativos instalados en los archivos de Gradle de Android.
4. **ionic cap open android:** Localiza el directorio /android y abre el proyecto nativo automáticamente en el entorno de desarrollo **Android Studio**.

Herramienta y generación del APK instalable:

El APK se compila mediante el sistema de construcción **Gradle** utilizando el Android SDK. Existen dos modalidades:

- **Por Terminal (Línea de Comandos):** Utilizando el wrapper de Gradle provisto en el proyecto:
  En Windows: cd android && .\gradlew.bat assembleDebug
  En Linux/macOS: cd android && ./gradlew assembleDebug
  El APK resultante se genera en android/app/build/outputs/apk/debug/app-debug.apk.
- **Por Android Studio:** Desde el menú *Build > Build Bundle(s) / APK(s) > Build APK(s)*.

PREGUNTA 8

Puntaje: 1,00 pt

Enunciado: *Explique la diferencia entre una página y un servicio en Ionic con Angular. Incluya un ejemplo de uso de cada uno.*

Respuesta emitida en la prueba:

«Pagina es la que nosotros creamos en el src/ de nuestra aplicacion y servicio la que viene los paquetes incluidos en ionic»

Diagnóstico del error y aspectos a mejorar:

Respuesta totalmente errónea. Tanto las páginas como los servicios son artefactos creados por el desarrollador dentro de src/app/. No distingue las capas arquitectónicas del software.

Respuesta Corregida y Fundamentación Técnica:

La diferencia se fundamenta en el principio de **Separación de Responsabilidades (*Separation of Concerns*)** de la arquitectura de software:

1. Página (Componente de Vista):
• **Definición:** Es un componente de Angular decorado con @Component() asociado a una ruta de navegación específica mediante el enrutador (routes).
• **Rol y responsabilidad:** Constituye la **Capa de Presentación e Interacción (UI)**. Posee plantilla visual HTML, estilos CSS/SCSS y clase TypeScript para responder a las interacciones del usuario (clicks, gestos, formularios). Maneja ganchos del ciclo de vida visual de Ionic (ionViewWillEnter).
• **Ejemplo:** ListadoPage (src/app/pages/listado/listado.page.ts), encargada de dibujar en pantalla las tarjetas de incidencias, la barra de búsqueda y el scroll infinito.

2. Servicio (Capa de Lógica y Datos):
• **Definición:** Es una clase de TypeScript decorada con @Injectable({ providedIn: 'root' }), que se comporta como un objeto único (*Singleton*) accesible mediante Inyección de Dependencias.
• **Rol y responsabilidad:** Constituye la **Capa de Lógica de Negocio y Acceso a Datos**. No tiene interfaz gráfica ni HTML. Se encarga de la comunicación con APIs externas vía HTTP, gestión del almacenamiento persistente y mantenimiento del estado global.
• **Ejemplo:** IncidenciasService (src/app/services/incidencias.service.ts), encargado de realizar las peticiones getIncidencias() y crearIncidencia() contra MockAPI, o CloudinaryService para la subida de fotos a la nube.

PREGUNTA 9

Puntaje: 1,00 pt

Enunciado: *¿Para qué se utiliza un servicio en Angular? Explique cómo compartiría información entre dos páginas de una aplicación Ionic.*

Respuesta emitida en la prueba:

«Se utiliza para poder dentro de una aplicacion compartir datos o mas fuentes que este enviando el usuario por ese medio, por ejemplo seria un Combobox que envie numeros de telefono o de un medio de pago donde se introduzca el numero de la tarjeta de credito o debito»

Diagnóstico del error y aspectos a mejorar:

Respuesta confusa e imprecisa. Confunde elementos de formulario (combobox, campos de texto) con la arquitectura de servicios y no explica los mecanismos reales de comunicación entre componentes.

Respuesta Corregida y Fundamentación Técnica:

1. ¿Para qué se utiliza un servicio en Angular?
Un servicio se utiliza para **centralizar la lógica de negocio, encapsular el acceso a datos** (peticiones HTTP con HttpClient), gestionar el estado de la aplicación y proporcionar funciones utilitarias reutilizables a múltiples componentes sin duplicar código.

2. ¿Cómo compartir información entre dos páginas de una aplicación Ionic?
Existen tres mecanismos técnicos bien definidos según la naturaleza del dato:
• **Mecanismo 1: Servicio Singleton Compartido con Reactividad (Recomendado):**
Se define un servicio con providedIn: 'root' que almacena el dato mediante **Signals** (signal()) o un BehaviorSubject de RxJS. La Página A inyecta el servicio y escribe el nuevo valor (ej. servicio.setSeleccion(dato)); la Página B inyecta la misma instancia del servicio y lee el valor reactivamente en su plantilla o clase.
• **Mecanismo 2: Parámetros de Ruta mediante Angular Router:**
Para identificadores o valores sencillos, se utiliza la URL con parámetros de ruta (ej. /incidencia/:id) o parámetros de consulta (/incidencias?estado=pendiente). La página receptora los extrae con route.snapshot.paramMap.get('id').
• **Mecanismo 3: Persistencia Local con Preferences:**
Cuando la información debe sobrevivir al reinicio de la aplicación o sincronizarse offline, la Página A persiste el dato con Preferences.set() y la Página B lo recupera con Preferences.get().

PREGUNTA 10

Puntaje: 1,00 pt

Enunciado: *Explique la diferencia entre las peticiones HTTP GET y POST en el contexto de consumo de APIs REST.*

Respuesta emitida en la prueba:

«la diferencia entre GET y POST es que GET obtiene de la API una respuesta cuando se envia en formato JSON y POST para cambiar ese recurso igual mediante un recurso JSON»

Diagnóstico del error y aspectos a mejorar:

Respuesta conceptualmente errónea. Las peticiones GET **no envían datos en formato JSON** (los parámetros viajan en la URL). Además, POST se utiliza para **crear** recursos nuevos, no para cambiarlos o modificarlos (para modificar se emplean PUT o PATCH).

Respuesta Corregida y Fundamentación Técnica:

En el estándar del protocolo HTTP (RFC 9110) y la arquitectura REST:

1. Método HTTP GET:
• **Propósito:** Se utiliza exclusivamente para **solicitar y consultar recursos** existentes en el servidor sin modificarlos.
• **Cuerpo de la petición (*Body*): No tiene cuerpo (*No Request Body*)**. Los filtros o datos viajan visibles en la propia URL como parámetros de consulta (*Query Parameters*, ej. /API?page=1&limit;=10) o de ruta (/API/15).
• **Propiedades HTTP:** Es un método **seguro** (no altera el estado del servidor) e **idempotente** (hacer la misma petición 1 o 100 veces produce el mismo resultado sin efectos colaterales). Puede ser almacenado en caché (*cacheable*).

2. Método HTTP POST:
• **Propósito:** Se utiliza para **crear un nuevo recurso** subordinado en el servidor o enviar datos para su procesamiento.
• **Cuerpo de la petición (*Body*):** Los datos viajan empaquetados dentro del cuerpo de la petición (Request Body), habitualmente en formato application/json (o multipart/form-data para archivos binarios).
• **Propiedades HTTP: No es seguro** (altera el estado del servidor creando un registro en la base de datos) y **no es idempotente** (enviar la misma petición POST 3 veces creará 3 registros independientes con diferentes IDs).

Aclaración de modificación: Para cambiar o actualizar un recurso existente se utiliza **PUT** (reemplazo completo) o **PATCH** (actualización parcial de campos), nunca POST.

CONCLUSIÓN Y RETROALIMENTACIÓN GENERAL

La revisión técnica realizada demuestra un dominio riguroso de la arquitectura de aplicaciones móviles con Ionic, Angular y Capacitor. Los errores iniciales de imprecisión en conceptos clave (como la distinción entre codificación Base64 y cifrado, el ciclo de vida con ngOnInit, la semántica HTTP REST y la separación de responsabilidades entre Páginas y Servicios) han sido corregidos con fundamentación teórica y aplicabilidad práctica directa en la solución desarrollada.